Encoding measurement-and-verification discipline as machine-readable skills so an AI agent enforces it

Rolf Elmér, Independent Researcher
rolf.elmer@aviverenavalis.se • ORCID 0009-0007-6735-3256

Preprint, version 1 — October 2026.
Extends a four-page version frozen on 18 August 2026 with evidence collected after the freeze (§3.6); every count is re-measured as of 22 October 2026.

Self-referential disclosure (read first). *This paper is written by the same class of AI agent it studies; its evidence is drawn from logged artifacts and version-control records, not the agent's self-report. §5 states what this establishes and what it does not. The author directed the work, reviewed and verified all content, and retains full responsibility for the text and its conclusions.*

Abstract

Demand flexibility in the heating of domestic housing is a developing avenue for adapting buildings to volatile green energy systems with intermittent production sources. Evaluating the effectiveness of demand-flexibility interventions requires robust measurement-and-verification (M&V) protocols — and how such protocols hold up under everyday research practice remains underexplored. Rigorous M&V depends on discipline that is easy to state and easy to violate silently: every number traceable to a source, figures produced by published code, data windows checked against experiment contamination, null results reported not buried. In a single-building home-energy management research programme we encoded this discipline as **machine-readable "agent skills"** — short documents an AI analysis agent loads automatically — then converted rules upheld by habit into **executable guards** that fail a build. The skills change agent behaviour by construction; the guards make the discipline *checkable*: a masking-coverage guard holds an explicit recorded decision for each of 112 readers of 13 sensitive streams, and a second enforces that a partly-delivered record cannot read as delivered, returning 75 latent inconsistencies on first run. Two results carry the argument: the gates repeatedly **failed the agent's own work** — a type gate rejected the contract of a guard committed nine minutes earlier — and the recurring defect they expose is nameable — a correct instruction with no code executing it. Evidence gathered after the first version sharpens both: a review of earlier repairs found two that had never taken effect, and a test that claimed to check a winter code path had in fact measured the calendar. The ceiling is explicit: these are *mechanical* defects, not the anti-HARK judgment that still requires a human, and a guard fires at the next review, not at delivery.

1 Divergence of development standards in LLM-assisted instrumentation

Buildings are a large share of energy use and emissions, and machine learning is now standard across climate-change mitigation, including building energy [1, 2]. Rigorous M&V of savings is itself a

modelling problem with established protocols [3, 4], and building studies increasingly lean on AI assistants. The risk is not that the assistant cannot compute — it is that it produces a *plausible* number, figure or claim not traceable to data, an error invisible until a reviewer, or nobody, catches it. The programme behind this paper is an occupied single-family house instrumented for demand-flexibility M&V; its methodology imposes by hand a precedence of measurement over documentation over model over memory, plus non-circular validation and experiment-window hygiene. These are the rules an eager assistant erodes fastest — and, as §3 shows, that a *human* erodes too, because a rule upheld by memory gives no signal when it stops being followed.

The stakes are set by a live development loop: the same agent that analyses the measurements also develops the price-responsive control system being measured — space heating, hot water and auxiliary loads scheduled against day-ahead electricity prices in the occupied house. It proposes, implements and evaluates controller changes faster than a human can audit each analysis by hand, so every savings estimate rests on the discipline holding at machine pace. The skills and guards below are what let the two coexist: daily controller iteration while every number reported about it stays traceable, contamination-checked and audit-grade by construction.

2 Encoding development standards as loadable skills and executable guards

We express each rule as a **machine-readable skill** — a short document with a triggering description an agent loads automatically when the task matches. Four encode the discipline studied here (212/71/68/77 lines): an operational standard; a research standard (epistemic ranking, non-circularity, honest limits); a data-loading standard flagging experiment-contaminated and frozen-logger rows; and a hot-water signal-semantics standard. The operational standard has grown from 124 to 212 lines since the first version.

Three mechanisms do the work, in increasing order of strength. **(i) Enforcement by construction:** the masked loader is mandatory in place of a bare `read_csv` and emits an audit trail — rows, span, experiment-window hits, freeze-trap status — pasted into each figure's methods note, so a compliant figure is audit-grade. **(ii) A rigor gate:** before results enter a manuscript, a fresh-context agent traces every number to its source and every figure to its code.

(iii) Executable guards. A skill still relies on the agent reading it; the stronger form is a check that turns a build red. A silent-degradation review stated the principle: the mask "exists and is tested, but before this guard nothing enforced that an analysis reading experiment-sensitive M&V data actually applies it — coverage depended on a developer remembering". Guards now encode former habits (§3.2), under a companion rule that a guard must be demonstrated to *fail* before it counts as deployed (§3.4).

3 Evidence

3.1 The loaded rules change what the agent produces; the gate catches what they miss. On its first smoke test the agent, unprompted, treated a partial-outage flag as a data-quality fact: it drew flagged days as lower-bound markers rather than readings and broke the plotted line across the gap. Coverage was 0.54–0.71 on flagged days against ∼0.98–1.0 on clean ones — anti-misleading-figure discipline arriving by construction, not by reminder. The gate then caught what the rules did not prevent: three fresh-context traces of the agent's drafting found four mechanical defects — a miscount, an unpropagated correction, off-by-one citations, a falsely-labelled "verbatim" block.

3.2 Masking coverage is decided rather than remembered, and enforced at commit time. The guard has blocked a real commit. The agent staged an analysis tool that read the water-meter history with no manifest entry, and the pre-commit hook stopped it (log name redacted):

```
[mv-masking] FAIL: 1 contract violation(s):  - UNDECIDED: ... reads sensitive
stem(s) ['⟨water-meter history⟩'] but has no entry in mv_masking_manifest.json
```

Eleven minutes later it landed in a masked revision. Complying produced the finding: masking removes a calibration day carrying an artificial 150 L draw injected into the very bins the tool's profile was built to correct, leaving 79 clean days of 88. The rule is that every reader of an experiment-sensitive log carries a recorded decision — masked, exempt, or gap — and the build fails on an undecided reader, a *masked* claim whose file does not mask, or a stale entry, "so the manifest cannot rot". The register stands at **112 readers of 13 streams: 73 analyses decided (27 masked, 43 exempt, 3 gaps) plus 39 excluded** with a reason. The three gaps are the register doing its job: a reader added on 11 October — a shadow daily total that is meant to replace a live figure — has no masked twin yet, and two exploratory analyses archived on 20 October mask through a precomputed day list the guard cannot verify and, in one table, not at all. Each is booked as a gap with a review date rather than exempted to keep the count at zero. Re-measuring on 1 October first gave one reader too many: one file was excluded twice, with two different reasons, and the guard's per-file lookup had silently kept only the second — the guard now fails on a duplicate entry. At the first version's freeze it held 43 readers of 9 streams, 12 of them gaps re-printed on every run so the debt could not expire quietly. All twelve have since closed — every one by a recorded exemption with a reason, two of them by explicit operator ruling with a review date, *none* by adding the mask (§5). A companion guard applies the same principle to records rather than readers: a partly-delivered item must not read as delivered, a check that returned **75 findings on first run, brought to 7** by two detector tunings. Unblocked, an experiment artefact would have entered a control model as occupant behaviour.

3.3 A named defect class with a corpus: intent stated, nothing executing it. The most frequent defect here is not a wrong calculation but a *correct* instruction with no code carrying it: eight instances in one day, across independent subsystems. A capitalised comment forbidding a register read had guarded nothing for three weeks — a one-off sweep, not a check — and a later analysis assumed the opposite rule. A watchdog computed a liveness rate into a column no verdict read, while its own verdict tested heartbeat age: it *recreated the defect it was built to fix, one level up.* A guard's docstring and its reason string both said "stale/missing" from its first revision while the code tested only `is None`, so a six-hour-old reading passed as fresh on a safety load. A significance criterion was unreachable by construction: eight permutations bound the attainable p at 1/9, against a 0.05 threshold. Two of the eight were found by *running the thing against a known outcome*, not by inspection: a dry run booked as a real repair, silently eating a daily budget; an exoneration credited to the wrong evidence. The class is mechanical, hence checkable; that is why naming it matters. §3.6 adds two post-freeze instances in which the stated intent was a *repair*.

3.4 The gates fail the agent's own work. This is the paper's central claim, and the hardest case for self-checking. Beyond the §3.2 block, two further gate refusals are on record.

#	Gate	What it judged	Outcome
1	Pre-push type contract	The consistency guard — committed by the agent nine minutes earlier — passed a mapping where the declared signature required a set. It worked via key membership but broke the contract.	Push failed; two-line fix, no logic change
2	Model fidelity gate	A replay tool built to quantify a model error was rejected at median 12 K: it assumed a measured inlet temperature while the running system applied a literature constant — measuring the agent's own wrong guess about which branch was live.	Rejected; diagnosis yielded a better method

Each is *mechanical*: a declared type, a numeric fidelity threshold. Neither required judgment about whether a conclusion was interesting — exactly the scope we claim. One property matters more than the count: the row-1 defect lived in the guard infrastructure itself, so the discipline caught its own author.

A guard counts only once it has fired *and* released against a known outcome. One verification step could never have emitted its own success string — an unquoted parenthesis made it a syntax error, and its expected value counted listener rows where the daemon opens three; both invisible until a real trial was forced. A coverage floor then failed a build at 89.3 % against its 95 % threshold; rather than lower the floor, ten pins took the untested branches to 100.0 %. And the promote-bundle rule, born of a release loop against a wrong baseline, is now code: seven pins, three proving it fires, four that it releases.

3.5 A finding must be able to look different. Before an observation may carry a claim, the programme now requires stating what it would have shown had the claim been false; if the answer is "the same", nothing was measured, however correctly read. Five instances surfaced in one day; three follow. An annotation column read empty both when a mask ran with no overlap and when it never ran. A green suite looked identical with one contaminated row and with none. Two MCMC chains returned $\hat{r} \approx 1.00$ both when converged and when both had settled in the same mode — four chains gave 1.77–3.73. Each reading was correct; none could have come out otherwise. The post-freeze record adds a deployment receipt of the same kind: a health check withholds one warning for the first two hours after a restart, and three receipts reading "no warnings" had all been taken inside that window.

3.6 Evidence after the 18 August freeze. Between 29 and 30 September the programme ran ten structured review passes over its control, measurement and deployment code, under a triage rule that a finding is reproduced before anything is repaired; the last three passes each used two read-only reviewer agents. They recorded **117 findings** across the ten passes, plus one surfaced by a dry-run rehearsal; the first pass counted only confirmed defects, the other nine all findings. Three results bear directly on the claims above.

(a) Repairs are a defect source, and a gate fires at the next review. In one pass, 8 of 11 findings lay in repairs written earlier the same day. In another, the review of the deployment tool for the home-automation hub's scripts — the only path from the repository to the running house — found that two repairs from that tool's previous review, eight weeks earlier, had never taken effect: a network-error wrapper with zero call sites, and a history scan that followed file renames but looked up every historical version under the *current* path, so no version committed before a rename could ever match. The commit message had described both as fixed, and no test exercised either — the §3.3 class, with a repair as the stated intent. The response, applied to every repair in that

pass, was procedural: a test that fails against the previous revision through the public entry point, never through the helper alone (a test calling the dead wrapper directly would have passed), and a mutation check [15]: removing each repair must turn its test red. All 34 did, in the logged run; two earlier survivors had exposed tests that had not reached the repaired line and were extended before it.

(b) A guard can be blind in a way that prints nothing. The same tool's flow-impact gate, which reports which automations reference a script before it is overwritten, recognised a call by an owner field that 11 of the house's 55 script-call cards lack — read once against the live configuration, disabled automations included. For 11 scripts — among them the hot-water controller and a heating fail-safe — it reported no referencing automation, and it reported zero by printing nothing. The blindness was found by running the parser read-only against the live configuration, not by inspection.

(c) A test that measured the calendar. A test asserted that *every* branch of the controller's per-zone explanation books the reference the savings ledger needs, using a January input. The controller reads the month from the wall clock by design — a wrong timestamp from the automation hub must not flip the season — so the input's month was ignored, and the test, added in early September, had only ever run in September: on the summer path, where the room needed no cooling and the decision fell through to a branch that does book the reference. At the month boundary on 1 October four tests in that file failed the pre-push gate, a textbook time-dependent test [16]. The test helper now pins the month to its input (a discriminating check: pinned to January, the three cooling tests fail; to July, they pass), which left one test red for a real reason: all three branches of the air-to-air zone's decision wrote their record without the reference, so the savings ledger skipped that zone with a warning whenever that branch was reached. Production logs held 41 such warnings on four days between 7 and 16 September; no one had acted on them. The test had claimed a winter check it never performed — a §3.5 reading that could not have come out otherwise — and only an accident of the calendar made it look different.

4 Related context

The pattern sits between reproducible computational research — community rules for traceable analysis [5], and evidence that many published notebooks fail to re-execute even when code is shared [6] — and LLM agents for science, which attempt end-to-end discovery [7], are benchmarked on data tasks [8], are asked whether they can review [9], and in shadow evaluations complete technical tasks yet fail open-ended research questions [14]. Neither the mechanism nor mechanical rigor-checking is new: reusable skill files are an established agent substrate [11, 12], and automated consistency checking predates LLMs [10]; concurrent work constrains workflow agents for reproducibility at the execution level [13]. The post-freeze response borrows from software testing, where mutation analysis measures whether a test can detect the fault it targets [15] and time- and order-dependent tests are a documented source of false confidence [16]. What is new is the *content and setting*: skills and guards encoding a programme's integrity rules, not task capability, enforced in a live, physically instrumented M&V setting, with caught violations — the agent's own included — as evidence.

5 Honest limitations

- **Self-referential evidence.** The rigor gate is the *same underlying model* checking itself. The §3.2, §3.4 and §3.6 cases beat a self-report because the judgments came from non-model

machinery — a hook, a type checker, a numeric threshold, a failing test — logged, not narrated. But the case *selection* is the agent's, and this paper is written by that model; whether models can review reliably is open [9].

- **Mechanical half only — sharpened, not softened, by the new evidence.** Every gate above decided a *formal* property: an absent manifest entry, a declared type, a transport status, a threshold, a call site. None could detect a hypothesis quietly changed after seeing the data, a null result left in a drawer, or a plausible-but-wrong framing of a real number. The §3.3 defect class is mechanical for the same reason — an intent with no carrier is decidable; whether it was *right* is not. Anti-HARK and anti-cherry-pick discipline remain human: the guards raise the floor, not the ceiling.

- **A gate fires at the next review, not at delivery.** The two repairs that never took effect (§3.6a) were delivered under the same discipline this paper describes and survived eight weeks. The discipline caught them, but its latency was a review cycle, and an unreviewed repair is unverified however confident its commit message.

- **A warning that fires is not a warning that is read.** The 41 production warnings of §3.6c were emitted, logged and ignored. A guard that reports into a channel nobody reads is a record, not a control.

- **A decision can be argued rather than enforced, and green history is not evidence of compliance.** The register's twelve gaps closed by exemption, not by masking (§3.2) — each on the argument that the measured quantity is invariant to the control regime, so a mask window could neither create nor hide the finding. That is a reclassification, and it may well be right; but the guard enforces only that a decision exists and is current, not that it is right. The §3.2 block is attested by a pre-commit transcript, not by version control — necessarily so, because a blocked commit leaves no trace in a repository's history. Every `git log` mention of that guard is green; reading that as compliance would invert the evidence. Our own first pass made that error. And one fired catch does not establish sensitivity across the other 111 readers.

- **N = 1 setting.** One programme, one agent stack, one building; worked examples, not a controlled evaluation. The counts (112, 75, 117, the eight-instance day) describe this repository's state at a stated date, not a rate — and the day's count is ours: a hard reading of a claimed nine reclassified one as a scope limitation. The §3.6 counts are traced in the numbers ledger but, unlike those of §2 and §3.2, not yet re-measured by the numbers check. The *structure* is transferable; the rules need re-authoring elsewhere.

- **Enforced traceability does not make a result driver-invariant.** A replication that reproduced archived verdicts to the decimal also showed a model-order verdict *flipping* when the driver was swapped. Mechanical traceability and scientific validity are different properties.

6 Conclusion

Encoding M&V discipline as machine-readable skills made a programme's conventions the *default* behaviour of an AI analysis agent; converting habitual rules into executable guards made them *checkable* — 112 sensitive-data readers now carry explicit decisions, three of them booked gaps. The gates failed the agent's own output, including the contract of a guard it had just written —

a discipline that only ever confirms its author is not a discipline — and they expose a nameable, mechanical, therefore checkable defect class: an intent with no code behind it, of which a repair that never takes effect is the most treacherous instance. The boundary is unchanged: an in-model gate enforces the mechanical half of rigor, with a latency of one review; the judgment half stays with the researcher.

Declaration of generative AI and AI-assisted technologies. This paper's subject is the encoding of M&V discipline for an AI agent, and its preparation used that same mechanism: during the preparation of this work the author used Claude (Anthropic) in agentic configurations for code review, analysis support and manuscript drafting, operating under the version-controlled verification discipline the paper describes — every numeric claim traced to a source record in an auditable numbers ledger, counts re-measured from the repository by a check that fails if one drifts, and drafts passed an independent traceability review before acceptance. The author reviewed, verified and edited all content and takes full responsibility for the publication. AI tools are not credited with authorship.

Competing interests. The author serves on the board of Kraftringen, a Swedish municipal energy company, on a political mandate; the household studied is a retail electricity customer of Kraftringen. Kraftringen provided no funding, data, or input to this work and had no role in its design, analysis, or conclusions. The price data used are public SE4 day-ahead market prices; savings are computed against the household's own metered consumption.

Availability. The skills are deposited at Zenodo, `https://doi.org/10.5281/zenodo.21420254` (all versions; CC BY 4.0); version 1.1.0, `https://doi.org/10.5281/zenodo.23074705`, holds the skills as of 1 October 2026. Release 1.2.0 of the same deposit adds this preprint, and its skill documents are the repository state of 12 October 2026 behind the line counts in §2. Logged artifacts behind §3 are available on request; controller code is project-internal under a staged release roadmap.

Version note. Version 1 extends the four-page version frozen on 18 August 2026: all counts re-measured (skills 124/71/68/71 → 212/71/68/77 lines; register 43 readers of 9 streams with 12 gaps → 112 of 13 with three, booked on 11 and 20 October, §3.2), §3.6 and two limitations added, the deployment receipt added to §3.5, two references added.

References

[1] D. Rolnick, P. L. Donti, L. H. Kaack, K. Kochanski, A. Lacoste, K. Sankaran, et al. Tackling climate change with machine learning. *ACM Computing Surveys*, 2022. doi:10.1145/3485128.

[2] L. H. Kaack, P. L. Donti, E. Strubell, G. Kamiya, F. Creutzig, and D. Rolnick. Aligning artificial intelligence with climate change mitigation. *Nature Climate Change*, 2022. doi:10.1038/s41558-022-01377-7.

[3] Y. Heo and V. M. Zavala. Gaussian process modeling for measurement and verification of building energy savings. *Energy and Buildings*, 2012. doi:10.1016/j.enbuild.2012.06.024.

[4] IPMVP Committee. *International Performance Measurement and Verification Protocol: Concepts and Options for Determining Energy and Water Savings, Volume I.* Technical report, OSTI, US Department of Energy, 2001. doi:10.2172/776003.

[5] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 2013. doi:10.1371/journal.pcbi.1003285.

[6] S. Samuel and D. Mietchen. Computational reproducibility of Jupyter notebooks from biomedical publications. *GigaScience*, 2024. doi:10.1093/gigascience/giad113.

[7] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. arXiv:2408.06292, 2024.

[8] K. Gu, R. Shang, R. Jiang, K. Kuang, R.-J. Lin, D. Lyu, et al. BLADE: Benchmarking language model agents for data-driven science. *Findings of EMNLP*, 2024. arXiv:2408.09667.

[9] W. Yuan, P. Liu, and G. Neubig. Can we automate scientific reviewing? *Journal of Artificial Intelligence Research*, 75:171–212, 2022. doi:10.1613/jair.1.12862.

[10] M. B. Nuijten, C. H. J. Hartgerink, M. A. L. M. van Assen, S. Epskamp, and J. M. Wicherts. The prevalence of statistical reporting errors in psychology (1985–2013). *Behavior Research Methods*, 48(4):1205–1226, 2016. doi:10.3758/s13428-015-0664-2.

[11] Z. Wang, J. Mao, D. Fried, and G. Neubig. Agent workflow memory. arXiv:2409.07429, 2024.

[12] Anthropic. Agent skills (open skill format), 2025. `https://github.com/anthropics/skills`.

[13] V. Sureshkumar. R-LAM: Reproducibility-constrained large action models for scientific workflow automation. arXiv:2601.09749, 2026 (unreviewed; concurrent work).

[14] P. Kirgis, S. Kapoor, A. Schwartz, S. Rabanser, et al. Can AI agents conduct open-ended AI research? Early evidence from two case studies. arXiv:2607.27191, 2026 (unreviewed preprint).

[15] Y. Jia and M. Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi:10.1109/TSE.2010.62.

[16] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 643–653, 2014. doi:10.1145/2635868.2635920.